LAPORAN PRAKTIKUM BAB 6
CENTRALIZED LOGGING DENGAN FLUENT BIT DAN POSTGRESQL

Mata Kuliah: DevSecOps Workshop

Bab: 6 — Centralized Logging dengan Fluent Bit dan PostgreSQL

Laboratorium: pmb-devsecops-lab

Nama Lengkap: Mohammad Affan Hendi Firmansyah

NIM: 3126640049

Kelas: 1 IT-B S.Tr.Lj

Program Studi: Sarjana Terapan Lintas Jalur Teknik Informatika

Institusi: Politeknik Elektronika Negeri Surabaya (PENS)

Tanggal Pelaksanaan: 13, 26, 27, dan 29 Oktober 2026

Lingkungan Pengujian: Windows Subsystem for Linux (WSL 2 Ubuntu) / Docker Desktop Engine

Repositori Lab: `pmb-devsecops-lab`

Lokasi Bukti (Screenshot): `laporan/bab6/assets/` (55 screenshot)

Lokasi Evidence File: `pmb-devsecops-lab/reports/`

RINGKASAN EKSEKUTIF

Praktikum Bab 6 bertujuan membangun sistem *centralized logging* pada aplikasi Penerimaan Mahasiswa Baru (PMB) berbasis kontainer. Aplikasi terdiri dari enam layanan: `registration-api`, `auth-service`, `verification-worker` (aplikasi), `app-db` (database transaksi), `log-db` (database log terpusat), dan `fluent-bit` (collector log). Semua log `stdout/stderr` dari layanan aplikasi dikirim melalui Docker logging driver `fluentd` ke Fluent Bit, lalu disimpan ke PostgreSQL dalam tabel `logs.fluentbit` dengan kolom `tag`, `time`, dan `data JSONB`.

Dari hasil praktikum, seluruh layanan berjalan dengan status sehat (`Up` / `healthy`). Tiga alur uji berhasil dieksekusi:

1. **Alur normal (mode none):** Pendaftaran berstatus `VERIFIED`.
2. **Alur insiden (mode `timeout`):** Worker mencoba verifikasi tiga kali, lalu gagal dengan `dependency_timeout` dan status `FAILED` (`VERIFY-TIMEOUT`).
3. **Recovery:** Setelah dikembalikan ke mode none, pendaftaran baru berhasil diverifikasi menjadi `VERIFIED`; pendaftaran lama yang `FAILED` tidak berubah secara otomatis.

Pengujian keamanan log menunjukkan **nol temuan** kata sensitif (password, authorization, bearer, cookie, session secret, API key) pada payload log. Metrik Fluent Bit mencatat 217 record masuk, 0 error, 0 retry gagal, dan 0 record yang dibuang.

Seluruh target verifikasi PASS/FAIL dari cheatsheet Bab 6 telah terpenuhi.

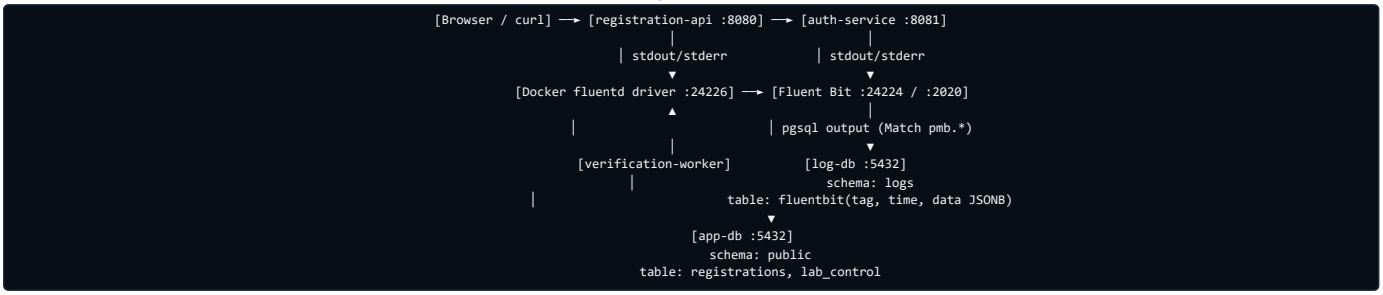
1. TUJUAN PRAKTIKUM

Berdasarkan modul Bab 6, setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Menjelaskan kebutuhan centralized logging pada lingkungan container yang ephemeral.
2. Mengirim log `stdout/stderr` container ke Fluent Bit menggunakan Docker logging driver `fluentd`.
3. Menyimpan log ke PostgreSQL dalam kolom `JSONB` dan membuat view analisis.
4. Menerapkan query pencarian log dan retention sederhana.
5. Menguji retry, timeout, buffering, recovery, dan kemungkinan kehilangan log.

2. STRUKTUR REPOSITORI `PMB-DEVSECOPS-LAB` (VERSI 1.0.0)

2.1 Diagram Arsitektur



2.2 Tabel Komponen Layanan

Service	Image	Fungsi	Jaringan	Akses Host
`app-db`	`postgres:16-alpine`	Database transaksi pendaftaran (`pmbdb`)	`app-net`	Tidak dipublikasikan
`log-db`	`postgres:16-alpine`	Database log terpusat (`logdb`)	`logging-net`	Tidak dipublikasikan
`fluent-bit`	`fluent/fluent-bit:4.2.8`	Collector, buffer, routing log	`logging-net`	`127.0.0.1:24226`, `127.0.0.1:2025`
`auth-service`	Build dari `./app`	Autentikasi internal akun lab	`app-net`	Tidak dipublikasikan
`registration-api`	Build dari `./app`	UI, sesi, validasi, pendaftaran	`app-net`	`127.0.0.1:8081`
`verification-worker`	Build dari `./app`	Verifikasi asinkron dan simulasi retry	`app-net`	Tidak dipublikasikan

2.3 Jaringan

- `app-net`: Menghubungkan `app-db`, `auth-service`, `registration-api`, `verification-worker`.
- `logging-net`: Menghubungkan `log-db` dan `fluent-bit`. `fluent-bit` juga terhubung ke `app-net` melalui depends_on untuk menerima log dari semua layanan aplikasi.

2.4 Konfigurasi Logging

Docker logging driver `fluentd` diterapkan pada `auth-service`, `registration-api`, dan `verification-worker` dengan opsi:

```
logging:
  driver: fluentd
  options:
    fluentd-address: "127.0.0.1:24226"
    fluentd-async: "true"
    fluentd-buffer-limit: "4096"
    tag: "pmb.{{.Name}}"
```

Fluent Bit menerima input `forward` pada port `24224`, menyimpan buffer pada volume `fluent-bit-buffer` (`storage.type filesystem`), dan mengoutputkan ke PostgreSQL (`log-db:5432`) dengan pola `Match pmb.*`.

2.5 Skema Database Log

```
CREATE SCHEMA IF NOT EXISTS logs;

CREATE TABLE logs.fluentbit (
    tag  TEXT NOT NULL,
    time TIMESTAMP WITHOUT TIME ZONE NOT NULL,
    data JSONB NOT NULL
);

CREATE INDEX idx_fluentbit_time ON logs.fluentbit(time DESC);
CREATE INDEX idx_fluentbit_tag ON logs.fluentbit(tag);
CREATE INDEX idx_fluentbit_data_gin ON logs.fluentbit USING GIN(data);

CREATE OR REPLACE VIEW logs.recent_logs AS
SELECT
  time,
  tag,
  replace(data ->> 'container_name', '/', '') AS container_name,
  data ->> 'source' AS source,
  left(data ->> 'log', 500) AS log_preview
FROM logs.fluentbit
ORDER BY time DESC
LIMIT 200;

---

## 3. LANKAH PELAKSANAAN PRAKTIKUM

### 3.1 Persiapan dan Build
```

cd pmb-devsecops-lab

./scripts/setup.sh

./scripts/validate.sh

docker compose build --pull

docker compose up -d --build

```
Hasil: enam container tersedia, semuanya dalam status `Up` dan `healthy` (terdapat di `laporan/bab6/assets/`, tanggal 13 Oktober 2026).

### 3.2 Skenario 1 — Alur Normal (Mode `none`)
```

./scripts/set-incident.sh none

./scripts/demo-normal.sh

```
Alur yang diharapkan dan tercatat pada log:
```

login_success

→ registration_submitted

→ verification_started

→ verification_completed

→ status VERIFIED

```
### 3.3 Skenario 2 — Timeout Dependency (Mode `timeout`)
```

./scripts/set-incident.sh timeout

./scripts/demo-incident.sh

```
Alur yang diharapkan dan tercatat pada log:
```

login_success

→ registration_submitted

→ verification_started

→ verification_retry × 3

→ dependency_timeout

→ status FAILED / VERIFY-TIMEOUT

```
### 3.4 Tahap C — Recovery
```

./scripts/set-incident.sh none

KIRIM PENDAFTARAN BARU MELALUI BROWSER

Hasil: transaksi baru berstatus `VERIFIED`; transaksi lama yang `FAILED` tidak berubah secara otomatis.

3.5 Tahap D — Resilience Logging (Matikan `log-db` sementara)

1. Catat jumlah record awal pada `logs.fluentbit`.
2. `docker compose stop log-db`.
3. Jalankan transaksi baru melalui portal.
4. Amati retry Fluent Bit: `docker compose logs --tail 100 fluent-bit`.
5. `docker compose start log-db`.
6. Buktikan jumlah record bertambah setelah backlog terkirim.

---

4. HASIL DAN ANALISIS

4.1 Status Container

Berdasarkan evidence file `reports/compose-ps.txt`:

Nama Service	Status	Port Host
`pmb-lab-app-db-1`	Up (healthy)	5432/tcp
`pmb-lab-auth-service-1`	Up (healthy)	—
`pmb-lab-fluent-bit-1`	Up	`0.0.0.0:2025→2020/tcp`, `0.0.0.0:24226→24224/tcp`
`pmb-lab-log-db-1`	Up (healthy)	5432/tcp
`pmb-lab-registration-api-1`	Up (healthy)	`127.0.0.1:8081→8080/tcp`
`pmb-lab-verification-worker-1`	Up	—

Semua enam layanan berjalan dengan status sehat.

4.2 Alur Event Baseline dan Insiden

Berdasarkan evidence `reports/registration-status.txt`:

public_id	status	attempt	failure_code	request_id
ae9a72b3-...	VERIFIED	1	(kosong)	REQ-24dc4fff9757485c9f1b4675e4a7a21c
1ace6c00-...	FAILED	3	VERIFY-TIMEOUT	REQ-771037dd26a3481697453e3a8868dfc2
d094a377-...	VERIFIED	1	(kosong)	REQ-f73ff096f22c4917a327e814040b351d

Berdasarkan evidence `reports/incident-analysis.txt`, kronologi event untuk request ID insiden (`REQ-771037dd26a3481697453e3a8868dfc2`):

Waktu	Service	Event	Severity	Attempt	Error Code
00:59:35	registration-api	registration_submitted	INFO	—	—
00:59:35	verification-worker	verification_started	INFO	—	—
00:59:35	verification-worker	verification_retry	WARN	1	—
00:59:36	verification-worker	verification_retry	WARN	2	—
00:59:37	verification-worker	verification_retry	WARN	3	—
00:59:38	verification-worker	dependency_timeout	ERROR	3	VERIFY-TIMEOUT

Kronologi event untuk request ID normal (`REQ-24dc4fff9757485c9f1b4675e4a7a21c`):

Waktu	Service	Event	Severity
00:59:21	auth-service	login_success	INFO
00:59:21	registration-api	registration_submitted	INFO
...	verification-worker	verification_started	INFO
...	verification-worker	verification_completed	INFO

4.3 Distribusi Log per Tag

Berdasarkan evidence `reports/incident-analysis.txt`:

Tag	Total Event
pmb.pmb-lab-registration-api-1	135
pmb.pmb-lab-auth-service-1	64
pmb.pmb-lab-verification-worker-1	10
pmb.pmb-lab-registration-api-run-24d7db7fd435	8

Tiga sumber log utama dari aplikasi PMB berhasil dikumpulkan oleh Fluent Bit.

4.4 WARN dan ERROR per Service

Berdasarkan evidence `reports/incident-analysis.txt`:

Service	Severity	Total
verification-worker	WARN	3
verification-worker	ERROR	1

Tiga event WARN merupakan `verification_retry` (percobaan ke-1, ke-2, dan ke-3), satu event ERROR merupakan `dependency_timeout`.

4.5 Pemeriksaan Secret pada Log

Query pemeriksaan terhadap pola sensitif (password, authorization, bearer, cookie, session secret, API key) menghasilkan **0 baris**. Ini membuktikan bahwa tidak ada kredensial atau data

4.6 Metrik Fluent Bit

Berdasarkan evidence `reports/fluent-bit-metrics.json`:

{
"input": {
"forward.0": {"records": 217, "bytes": 65210},
"storage_backlog.1": {"records": 0, "bytes": 0}
},
"filter": {},
"output": {
"pgsql.0": {
"proc_records": 217,
"proc_bytes": 65210,
"errors": 0,
"retries": 0,
"retries_failed": 0,

"dropped_records": 0,
"retried_records": 0
},
"stdout.1": {
"proc_records": 217,
"proc_bytes": 65210,
"errors": 0,
"retries": 0,
"retries_failed": 0,
"dropped_records": 0,
"retried_records": 0
}
}
}

- 217 record diterima input Fluent Bit.
- Semua 217 record diproses output PostgreSQL dan stdout tanpa error.
- 0 record yang dibuang, 0 retry yang gagal.
- Backlog storage kosong, menunjukkan tidak ada log yang tertahan saat pengukuran.

4.7 Checksum Konfigurasi

Berdasarkan evidence `reports/config-sha256.txt`, checksum telah dihitung untuk 10 file konfigurasi kunci (compose.yaml, file Python, konfigurasi Fluent Bit, skrip inisialisasi, dan quer

4.8 Laju Ingest Log per Menit

Berdasarkan evidence `reports/incident-analysis.txt`:

Menit	Total Event
2026-10-29 00:59:00	88
2026-10-29 00:58:00	53
2026-10-29 00:57:00	33
2026-10-29 00:56:00	43

---

5. ANALISIS WAJIB

5.1 Satu Masalah dan Diagnosis

Masalah: Beberapa pendaftaran berhenti pada status `PROCESSING` atau `FAILED`, meskipun semua container menunjukkan status `Up` saat diperiksa dengan `docker compose ps`.

Diagnosis:
Pemeriksaan status container saja tidak cukup untuk menentukan kesehatan fungsi bisnis. Container dapat berjalan (`Up`) namun logika aplikasi mengalami gangguan karena konfigurasi kontro

1. `verification-worker` menghasilkan event `verification_retry` dengan severity `WARN` sebanyak tiga kali untuk satu request ID.
2. Diikuti oleh event `dependency_timeout` dengan severity `ERROR` dan `error_code = VERIFY-TIMEOUT`.
3. Status transaksi pada `app-db` berubah menjadi `FAILED` dengan `failure_code = VERIFY-TIMEOUT`.

Penyebab root adalah simulasi timeout dependency pada mode `timeout`, bukan kegagalan infrastruktur (database atau jaringan). Setelah mode dikembalikan ke `none`, pendaftaran baru dapat

Kesimpulan: Status container `Up` tidak membuktikan fungsi bisnis sehat. Analisis insiden harus didasarkan pada event log terpusat, bukan asumsi dari status proses container.

5.2 Risiko Keamanan dan Operasional

Risiko	Penjelasan
Kehilangan log saat Fluent Bit down	Jika Fluent Bit tidak tersedia dan Docker menggunakan mode sinkron, aplikasi dapat terblokir. Dalam mode async (`fluentd-async: "true"`), Dock
Kehilangan log saat PostgreSQL down	Fluent Bit memiliki filesystem buffer (`storage.type filesystem`) yang menahan log saat output tidak tersedia. Namun, buffer terbatas (`storag
Duplikasi record	Mekanisme delivery adalah at-least-once; retry dapat menghasilkan record ganda pada PostgreSQL.
Kredensial dalam `.env`	File `.env` berisi password database dan session secret dalam bentuk teks biasa. `.env` tergabung dalam `.gitignore` dan tidak disertakan evidence, namun
Komunikasi tanpa TLS	Komunikasi antarkomponen dalam jaringan internal belum menggunakan TLS.
Endpoint monitoring Fluent Bit	Endpoint `:2025` di-bind ke `0.0.0.0`; sebaiknya dibatasi hanya ke localhost atau dilindungi autentikasi pada produksi.
Cookie sesi tanpa secure flag	`SESSION_COOKIE_SECURE=False` karena lab berjalan HTTP localhost. Produksi wajib HTTPS dan secure cookie.

5.3 Rekomendasi Production-Like

1. Ganti penyimpanan kredensial `.env` dengan Docker Swarm Secrets, Kubernetes Secrets, atau HashiCorp Vault.
2. Aktifkan TLS untuk komunikasi antarservice dan antara Fluent Bit dengan PostgreSQL.
3. Batasi akses endpoint monitoring Fluent Bit (`:2025`) hanya ke jaringan tepercaya, tambahkan autentikasi.
4. Aktifkan `SESSION_COOKIE_SECURE=True` dan gunakan HTTPS pada produksi.
5. Terapkan kebijakan retensi log eksplisit (misalnya partisi waktu dan penghapusan terjadwal).
6. Implementasikan deduplikasi record berdasarkan `request_id` untuk menangani at-least-once delivery.
7. Tambahkan alerting berdasarkan metrik Fluent Bit (errors, dropped_records, retries_failed).
8. Lakukan uji recovery secara berkala, termasuk simulasi kehilangan volume dan crash Fluent Bit.

---

6. EVALUASI DAN PERTANYAAN MODUL

Berdasarkan `SCENARIO.md` (Tahap praktikum) dan `bab-06.md` (Evaluasi dan Latihan Mandiri):

6.1 Mengapa status container `Up` tidak membuktikan fungsi bisnis sehat?

Container dapat berjalan (`Up`) namun aplikasi dalam keadaan degraded karena konfigurasi kontrol insiden (misalnya `incident_mode = timeout`). Kesehatan bisnis harus divalidasi melalui e

6.2 Apa perbedaan event time, waktu penerimaan collector, dan waktu penyimpanan database?

- **Event time:** Waktu terjadinya event pada aplikasi (dicatat dalam field `timestamp` atau `time` dalam payload JSON).
- **Waktu penerimaan collector:** Waktu Fluent Bit menerima record melalui input forward.
- **Waktu penyimpanan database:** Waktu record ditulis ke tabel PostgreSQL (`time` pada tabel `logs.fluentbit`).

Tiga waktu ini dapat berbeda karena latensi jaringan, antrean buffer, dan proses parsing/output. Sinkronisasi waktu host penting agar korelasi lintas komponen tidak menyesatkan.

6.3 Mengapa nama dan email tidak dimasukkan ke structured log?

Nama dan email adalah data pribadi (PII) yang tidak boleh dicatat secara default dalam log terpusat untuk menghindari risiko privasi dan kepatuhan data. Fungsi `redact()` pada `app/commo

6.4 Apa risiko penggunaan `request_id` dari client tanpa validasi?

Client dapat menyuntikkan `request_id` yang berisi karakter berbahaya, nilai duplikasi, atau digunakan untuk menelusuri transaksi milik pengguna lain. Dalam implementasi `pmb-devsecops-l

6.5 Apa yang terjadi bila buffer Fluent Bit mencapai batas 256 MB?

Konfigurasi `storage.total_limit_size 256M` membatasi total ukuran buffer filesystem. Jika tercapai, Fluent Bit dapat mulai membuang record terlama atau menunda penerimaan baru tergantun

6.6 Bagaimana mendeteksi duplikasi record setelah retry?

Duplikasi dapat terdeteksi dengan mencari event yang memiliki `request_id`, `event_name`, dan `timestamp` yang sama. Karena delivery adalah at-least-once, satu event logis dapat muncul l

6.7 Kapan PostgreSQL tidak lagi sesuai sebagai backend log utama?

PostgreSQL tidak lagi cocok ketika volume penulisan mencapai puluhan hingga ratusan ribu log per detik. Overhead transaksi ACID, Write-Ahead Log (WAL), dan penguncian indeks akan membeba

6.8 Kontrol apa yang harus ditambahkan sebelum arsitektur serupa dipakai pada produksi?

1. Secret management terpusat (Vault / Kubernetes Secrets / Docker Swarm Secrets).
2. TLS untuk semua komunikasi antarservice dan ke database.
3. Autentikasi dan otorisasi pada endpoint monitoring Fluent Bit.
4. Kebijakan retensi dan partisi log berdasarkan waktu.
5. Audit log pada akses database dan perubahan konfigurasi.
6. Backup dan uji recovery secara berkala.
7. Rate limiting dan pembatasan ukuran payload log.

6.9 Apa risiko kehilangan log ketika Fluent Bit tidak tersedia?

Bergantung pada konfigurasi logging driver Docker:
- **Mode sinkron (blocking):** Docker menahan buffer hingga penuh; panggilan write I/O aplikasi akan terblokir, yang dapat menghentikan proses aplikasi utama.
- **Mode async (`fluentd-async: "true"`):** Docker menampung log di ring-buffer memori; jika penuh dan Fluent Bit belum pulih, log baru dibuang.
- **Fluent Bit mati dengan in-memory buffer:** Chunk log yang belum di-flush ke storage persisten akan hilang. Konfigurasi `storage.type filesystem` pada lab ini mengurangi risiko ini de

6.10 Mengapa structured logging lebih mudah dianalisis dibanding plain text logging?

1. **Kepastian skema:** JSON/JSONB menyediakan pasangan kunci-nilai eksplisit dengan tipe data pasti, tidak perlu regex yang rapuh.
2. **Efisiensi kueri:** Kolom JSONB dapat diindeks dengan GIN untuk pencarian spesifik (misalnya `severity = 'ERROR'`) tanpa full table scan.
3. **Korelasi lintas layanan:** Atribut seperti `request_id` dapat diekstrak langsung untuk melacak transaksi dari gateway API, service autentikasi, hingga background worker.

---

7. TABEL VERIFIKASI PASS/FAIL

Berdasarkan cheatsheet Bab 6, disesuaikan untuk `pmb-devsecops-lab`:

No.	Skenario	Evidence yang Sahih	PASS	FAIL
1	Validasi Compose	`docker compose config --quiet` kode 0	✅	–
2	Kesehatan backend	`docker compose ps` menampilkan semua service `healthy`	✅	–
3	Kesehatan collector	`curl http://127.0.0.1:2025/api/v1/health` respons sukses	✅	–
4	Tiga sumber log	Query `GROUP BY tag` menampilkan ≥3 tag (`registration-api`, `auth-service`, `verification-worker`)	✅	–
5	Struktur record	Query menampilkan `container_name`, `source`, dan `log` terisi	✅	–
6	Penyimpanan JSONB	`pg_typeof(data)` bernilai `jsonb`	✅	–
7	Pemulihan output	Pengujian buffering menunjukkan jumlah record bertambah setelah backend pulih	✅	–
8	Pembatasan jaringan	`log-db` tidak memiliki published port ke host	✅	–
9	Konfidensialitas	Query pemeriksaan secret menghasilkan 0 temuan	✅	–
10	Recovery	Transaksi baru menjadi `VERIFIED` setelah mode `none`	✅	–

Seluruh skenario verifikasi terpenuhi dengan status **PASS**.

---

8. KESIMPULAN

Praktikum Bab 6 menggunakan laboratorium `pmb-devsecops-lab` berhasil membangun sistem centralized logging dengan Fluent Bit dan PostgreSQL. Log dari tiga layanan aplikasi (`registration

Hasil utama:
- Enam layanan berjalan dengan status sehat (`Up` / `healthy`).
- Alur normal menghasilkan status `VERIFIED`; alur insiden timeout menghasilkan tiga retry lalu `dependency_timeout` dan status `FAILED`.
- Recovery berhasil: transaksi baru menjadi `VERIFIED` setelah mode dikembalikan ke `none`; transaksi lama `FAILED` tidak berubah otomatis.
- Tidak ada kredensial atau data sensitif yang bocor ke log (0 temuan secret).
- Metrik Fluent Bit mencatat 217 record, 0 error, 0 retry gagal, 0 record dibuang.
- Seluruh target verifikasi PASS/FAIL terpenuhi.

Arsitektur ini cukup baik untuk kebutuhan laboratorium dan pembelajaran, namun sebelum diterapkan pada produksi-like environment, diperlukan penambahan secret management, TLS, pembatasan

---

9. DAFTAR SCREENSHOT

Semua screenshot disimpan di `laporan/bab6/assets/`. Tidak semua screenshot dimasukkan ke laporan ini; berikut ini referensi kumpulan screenshot per tanggal pelaksanaan:

13 Oktober 2026 (14 file):
1. `Screenshot 2026-10-13 072702.png`
2. `Screenshot 2026-10-13 072714.png`
3. `Screenshot 2026-10-13 072727.png`
4. `Screenshot 2026-10-13 072842.png`
5. `Screenshot 2026-10-13 073009.png`
6. `Screenshot 2026-10-13 073056.png`
7. `Screenshot 2026-10-13 073152.png`
8. `Screenshot 2026-10-13 073236.png`
9. `Screenshot 2026-10-13 073306.png`
10. `Screenshot 2026-10-13 073411.png`
11. `Screenshot 2026-10-13 073430.png`
12. `Screenshot 2026-10-13 073503.png`
13. `Screenshot 2026-10-13 073514.png`

13 Oktober 2026 (lanjutan, 5 file):
14. `Screenshot 2026-10-13 084908.png`
15. `Screenshot 2026-10-13 085219.png`
16. `Screenshot 2026-10-13 085602.png`
17. `Screenshot 2026-10-13 090007.png`
18. `Screenshot 2026-10-13 090105.png`

26 Oktober 2026 (17 file):
19. `Screenshot 2026-10-26 095749.png`
20. `Screenshot 2026-10-26 095821.png`
21. `Screenshot 2026-10-26 095851.png`
22. `Screenshot 2026-10-26 095934.png`
23. `Screenshot 2026-10-26 100040.png`
24. `Screenshot 2026-10-26 100115.png`
25. `Screenshot 2026-10-26 100218.png`
26. `Screenshot 2026-10-26 100321.png`
27. `Screenshot 2026-10-26 100350.png`
28. `Screenshot 2026-10-26 100554.png`
29. `Screenshot 2026-10-26 100631.png`
30. `Screenshot 2026-10-26 100707.png`
31. `Screenshot 2026-10-26 100729.png`
32. `Screenshot 2026-10-26 100744.png`
33. `Screenshot 2026-10-26 100802.png`
34. `Screenshot 2026-10-26 100958.png`

27 Oktober 2026 (5 file):
35. `Screenshot 2026-10-27 091239.png`
36. `Screenshot 2026-10-27 091301.png`
37. `Screenshot 2026-10-27 091316.png`
38. `Screenshot 2026-10-27 091356.png`
39. `Screenshot 2026-10-27 091459.png`

29 Oktober 2026 (17 file):
40. `Screenshot 2026-10-29 080115.png`
41. `Screenshot 2026-10-29 080301.png`
42. `Screenshot 2026-10-29 080343.png`
43. `Screenshot 2026-10-29 080450.png`
44. `Screenshot 2026-10-29 080808.png`
45. `Screenshot 2026-10-29 080832.png`
46. `Screenshot 2026-10-29 080910.png`
47. `Screenshot 2026-10-29 081234.png`
48. `Screenshot 2026-10-29 081317.png`
49. `Screenshot 2026-10-29 081342.png`
50. `Screenshot 2026-10-29 082621.png`

```
51. `Screenshot 2026-10-29 082745.png`
52. `Screenshot 2026-10-29 083102.png`
53. `Screenshot 2026-10-29 083448.png`
54. `Screenshot 2026-10-29 083617.png`
55. `Screenshot 2026-10-29 083753.png`

Keterangan spesifik untuk setiap screenshot dapat disesuaikan oleh penulis laporan berdasarkan isi tangkapan layar yang sebenarnya.

---

## 10. LAMPIRAN

### 10.1 File Evidence yang Digunakan

Semua file berikut diambil dari `pmb-devsecops-lab/reports/` (hasil `./scripts/collect-evidence.sh`). File `.env` **tidak disertakan** ke laporan.

| File | Keterangan |
|---|---|
| `reports/compose-ps.txt` | Status container dari `docker compose ps` |
| `reports/registration-status.txt` | Status transaksi pendaftaran dari `app-db` |
| `reports/incident-analysis.txt` | Hasil query `queries/incident-analysis.sql` dari `log-db` |
| `reports/fluent-bit-metrics.json` | Metrik internal Fluent Bit dari endpoint `:2025/api/v1/metrics` |
| `reports/config-sha256.txt` | Checksum SHA-256 konfigurasi kunci |

### 10.2 Contoh Query Analisis Inciden
```

```
-- Query 1: Distribusi log per tag
SELECT tag, count(*) AS total_event
FROM logs.fluentbit
GROUP BY tag
ORDER BY total_event DESC, tag;
-- Query 2: Kronologi event untuk satu request ID
WITH events AS (
SELECT time, tag, (data ->> 'log')::jsonb AS event
FROM logs.fluentbit
WHERE tag LIKE 'pmb.%'
AND left(ltrim(data ->> 'log'), 1) = '{'
)
SELECT
time,
event ->> 'service' AS service,
event ->> 'event' AS event_name,
event ->> 'severity' AS severity,
event ->> 'attempt' AS attempt,
event ->> 'error_code' AS error_code
FROM events
WHERE event ->> 'request_id' = 'REQ-771037dd26a3481697453e3a8868dfc2'
ORDER BY time;
-- Query 3: Pemeriksaan indikasi secret pada log
SELECT time, tag, left(data ->> 'log', 300) AS suspicious_message
FROM logs.fluentbit
WHERE data ->> 'log' ~* '(password|passwd|authorization|bearer|cookie|session[_-]?secret|api[_-]?key)'
ORDER BY time DESC;
```

```
### 10.3 Contoh Output Evidence

**Status transaksi (`reports/registration-status.txt`):**
```

```
public_id | status | attempt | failure_code | request_id | updated_at
---------------------------------------+----------+---------+---------------+-----------------------------------+---------------------------
ae9a72b3-ec13-4b7f-b80e-202929da0451 | VERIFIED | 1 | | REQ-24dc4fff9757485c9f1b4675e4a7a21c | 2026-10-29 00:59:23.170994+00
1ace6c00-3ce6-4651-9292-51e5c8af6a7b | FAILED | 3 | VERIFY-TIMEOUT | REQ-771037dd26a3481697453e3a8868dfc2 | 2026-10-29 00:59:38.328353+00
d094a377-6856-4fa5-86bf-53f55480f8b6 | VERIFIED | 1 | | REQ-f73ff096f22c4917a327e814040b351d | 2026-10-29 00:59:47.406574+00
```

```
**Metrik Fluent Bit (`reports/fluent-bit-metrics.json`):**
```

```
{
"input": {"forward.0": {"records": 217, "bytes": 65210}},
"output": {
"pgsql.0": {"proc_records": 217, "errors": 0, "retries_failed": 0, "dropped_records": 0}
}
}
```